Evidence Document

Course: Software Engineering

Project: SuperEduAI-IBCM

Group: 8

Task: 1

Name: Albana Hajraj

Date: 10/29/2026

Version 1.0; repository URL: https://github.com/IBCM-SuperEduAI-2026/SuperEduAI.

Introduction

Purpose of this document. This document is the evidence for Group 8, task 1 of the IBCM SuperEduAI project: setting up the GitHub repository and branches. SuperEduAI is an AI-powered educational platform built by nine student groups at the same time. For nine groups to work without overwriting each other's code, the project needs a shared home, clear folders, controlled access and rules for how changes are made. I set these up and describe each part below, with a screenshot as proof.

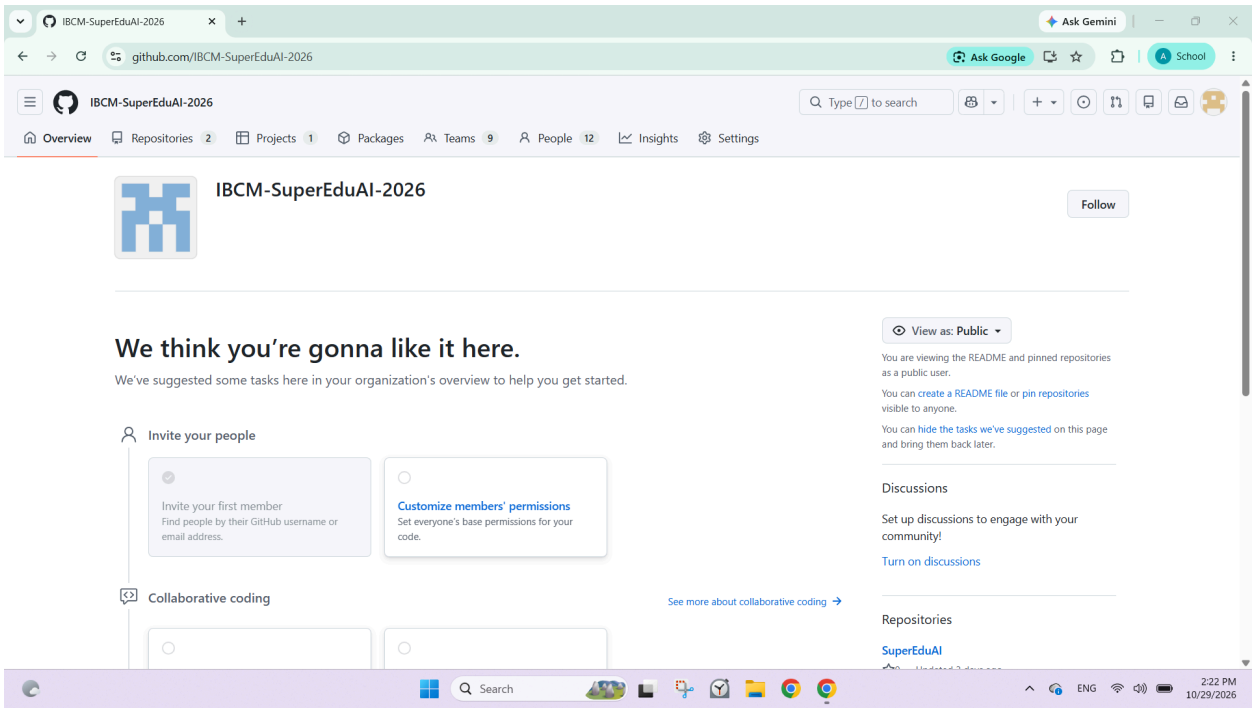


Fig. 1: The IBCM-SuperEduAI-2026 GitHub organization with the SuperEduAI repository.

This shows the GitHub organization `IBCM-SuperEduAI-2026`, which owns the project, with the SuperEduAI repository listed. I used an organization instead of a personal account so that ownership belongs to the whole class project and every group can be given access in a controlled way.

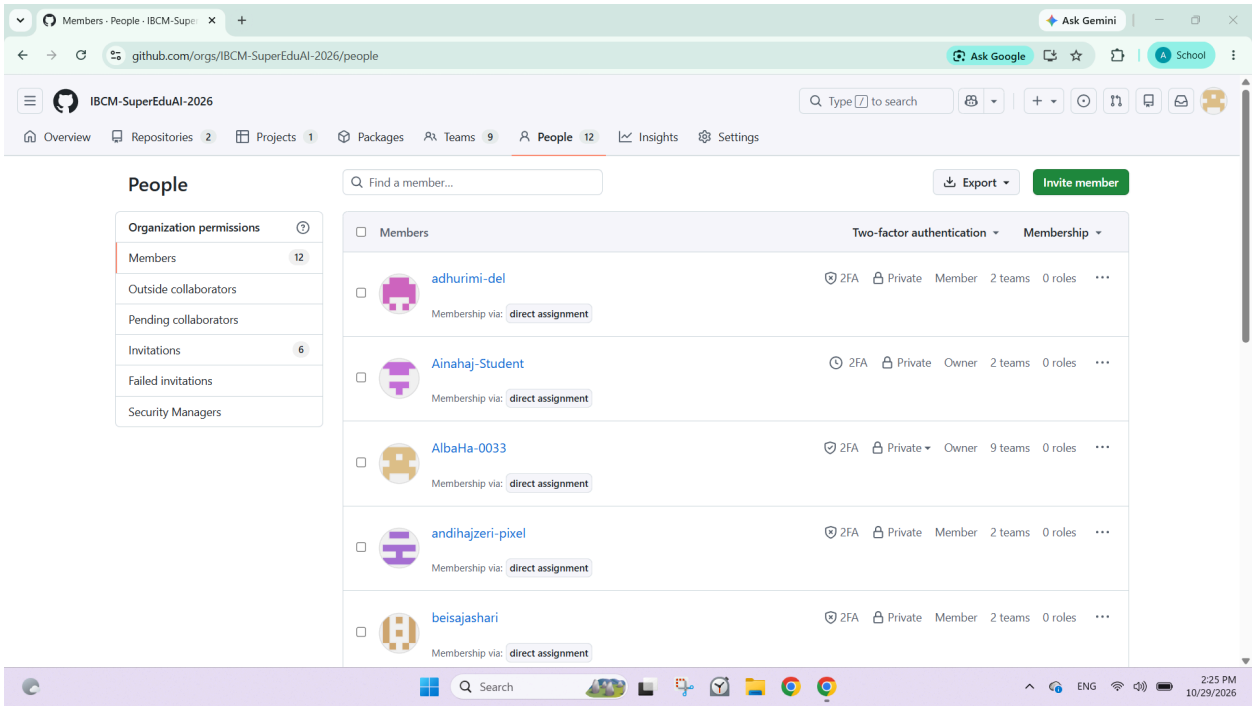


Fig. 2: Organization members.

This shows everyone invited to the organization who has accepted: 12 people and other that were invited but did not accept the invitation. Membership is the first level of access. It means a person belongs to the project, but they can only change things in the repository once their team has been given permission (Figures 4 and 5).

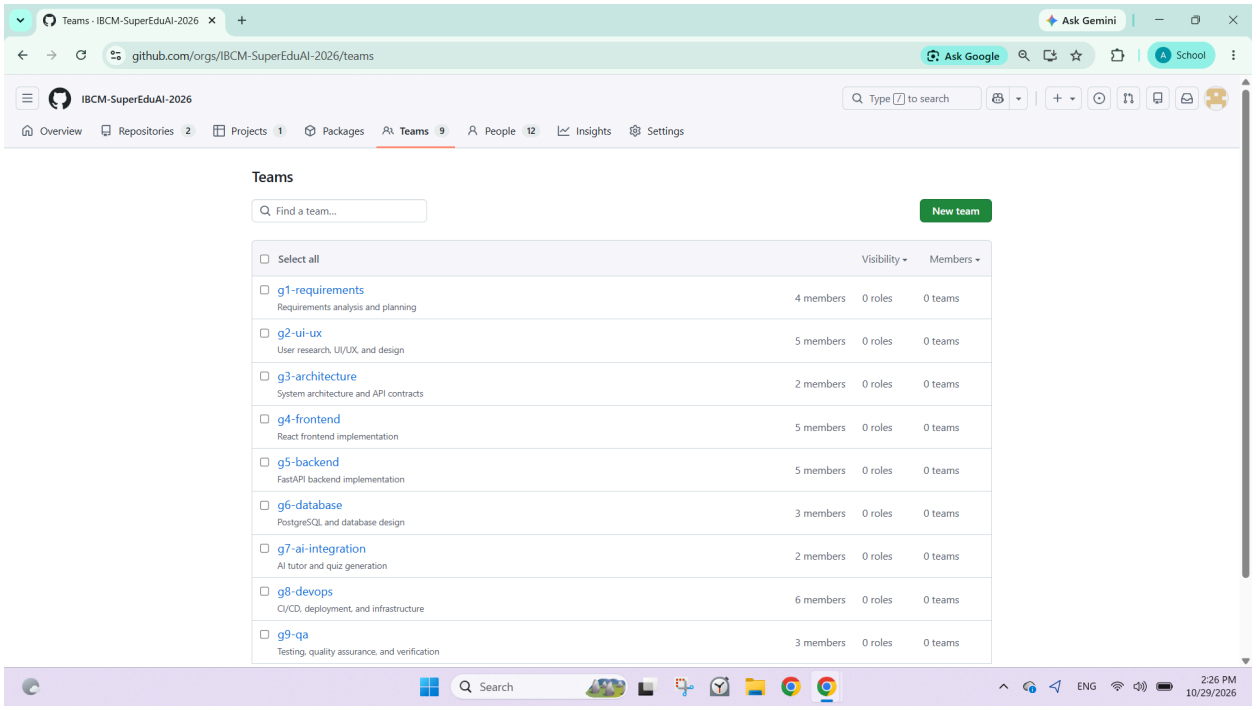


Fig 3: Teams G1 to G9 in the organization.

I created one team per project group, matching the nine groups in the course briefing (for example G1 Requirements and Planning, G4 Frontend, G8 DevOps). With teams, access is managed per group instead of per person: when someone joins a group, I add them to one team and they get the right access.

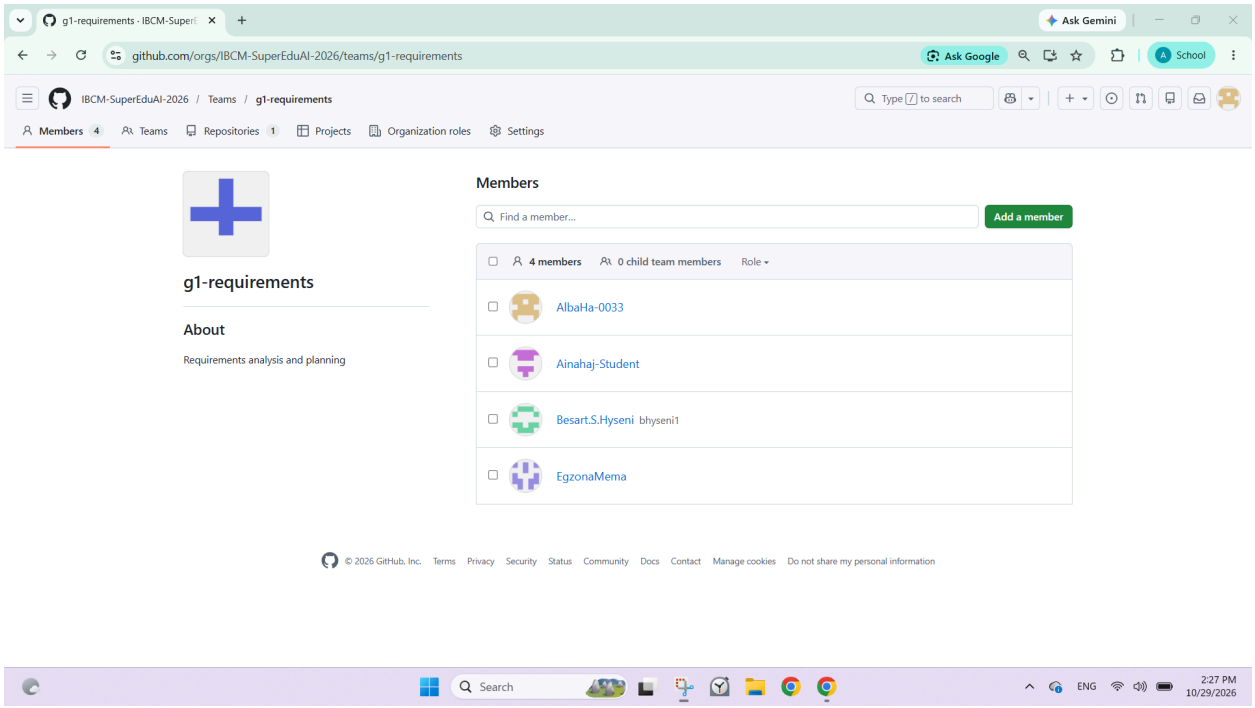


Fig 4: Members of team G 1

This shows who is in team g1-requirements. It links each student to their group, so it is visible who is responsible for which part of the platform. All other teams are set up the same way.

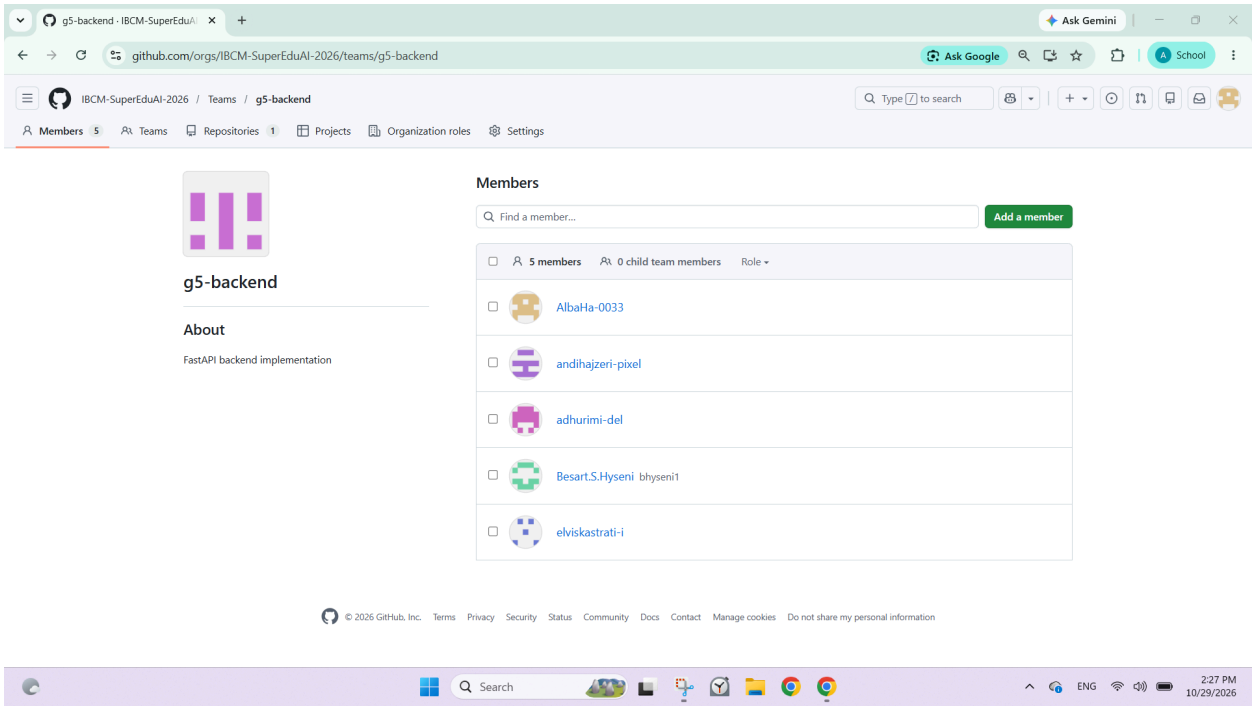


Fig 4: Members of team G 5

This shows who is in team g5-backend. It links each student to their group, so it is visible who is responsible for which part of the platform. All other teams are set up the same way.

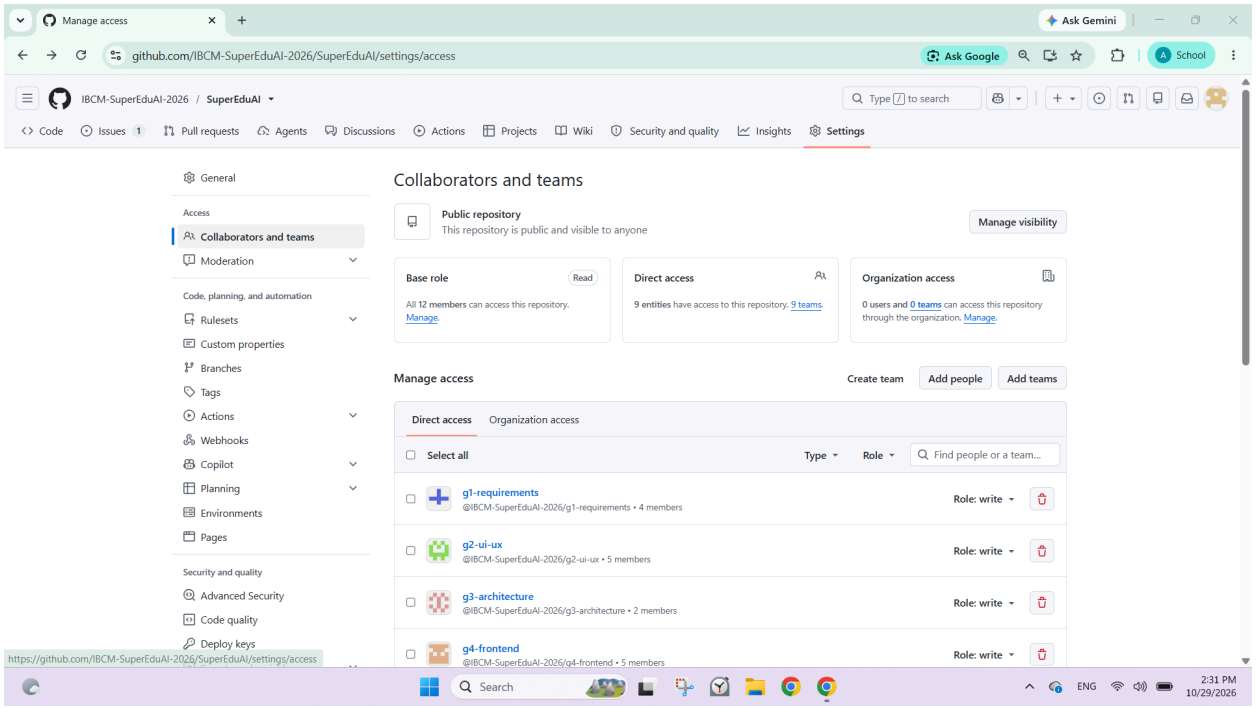


Fig 5a: Repository access for each team (Settings, Collaborators and teams).

Under Settings -> Collaborators and teams, each team has Write access to the repository. Write lets members create branches, commit and open pull requests. It does not let them change repository settings or delete the repository, which stay with the organization owners. This is the minimum access needed to contribute.

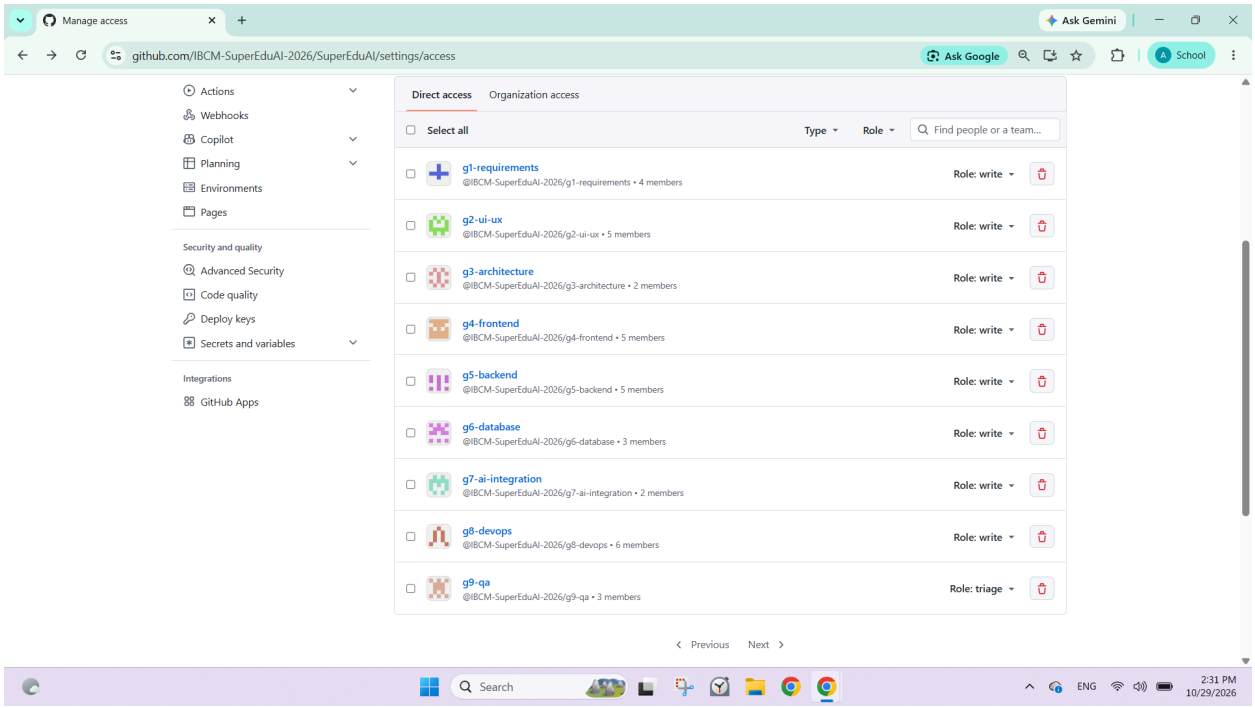


Fig 5b: Repository access for each team (Settings, Collaborators and teams).

Under Settings -> Collaborators and teams, each team has Write access to the repository. Write lets members create branches, commit and open pull requests. It does not let them change repository settings or delete the repository, which stay with the organization owners. This is the minimum access needed to contribute. Also the group 9 have the role of triage This gives them full control over managing bugs and issues like labeling, assigning tasks, and keeping the backlog clean without giving them direct write access to the codebase. It keeps our QA process moving fast while keeping the code secure.

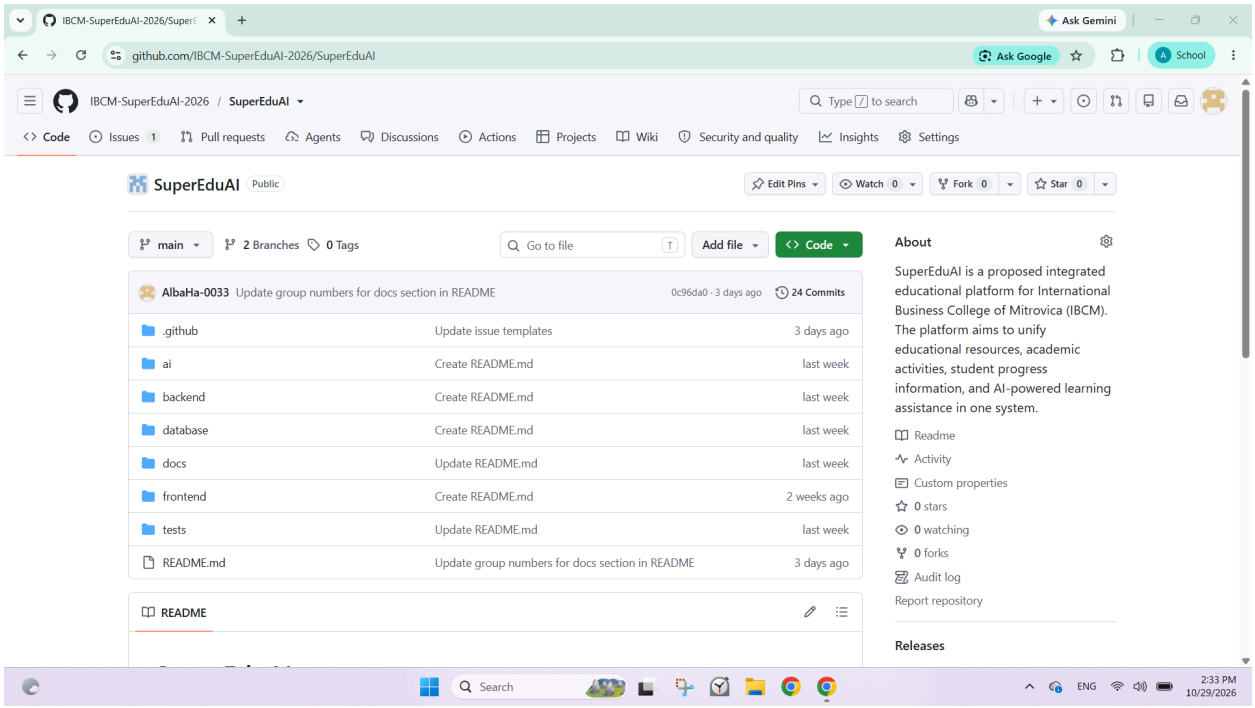


Fig 6: Folder structure of the SuperEduAI repository on `main`

The repository follows the structure set in the course briefing: frontend (React user interface), backend (FastAPI services), database (schema and migrations), ai (AI tutor and quiz engine), docs (requirements, architecture, evidence), tests (QA) and .github (templates and automation). Each folder belongs mainly to one group, so people know where their work goes and groups rarely edit the same files.

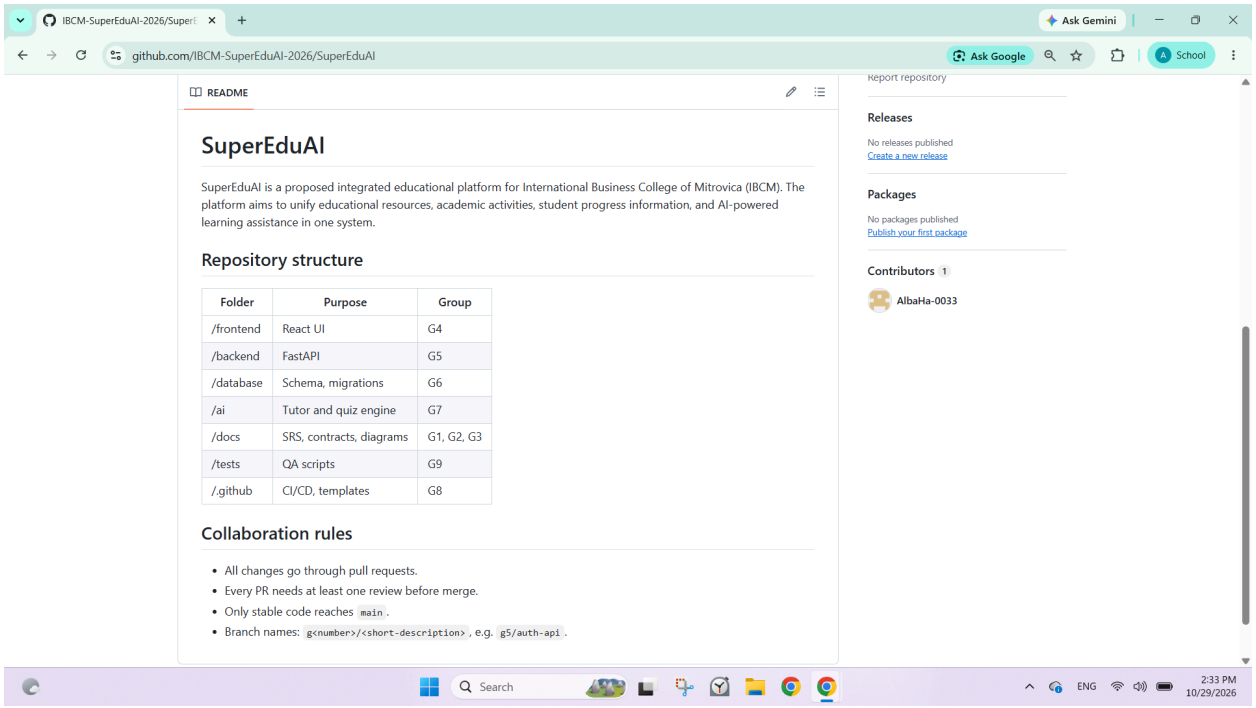


Fig 7: The project README.

The README is the first thing a new contributor sees. It describes the project, the folder layout and the collaboration rules: all changes through pull requests, a review before every merge, only stable code in `main`, and branch names in the form `g<number>/<short-description>`, for example `g5/auth-api`. Writing the rules down means everyone can follow the same process.

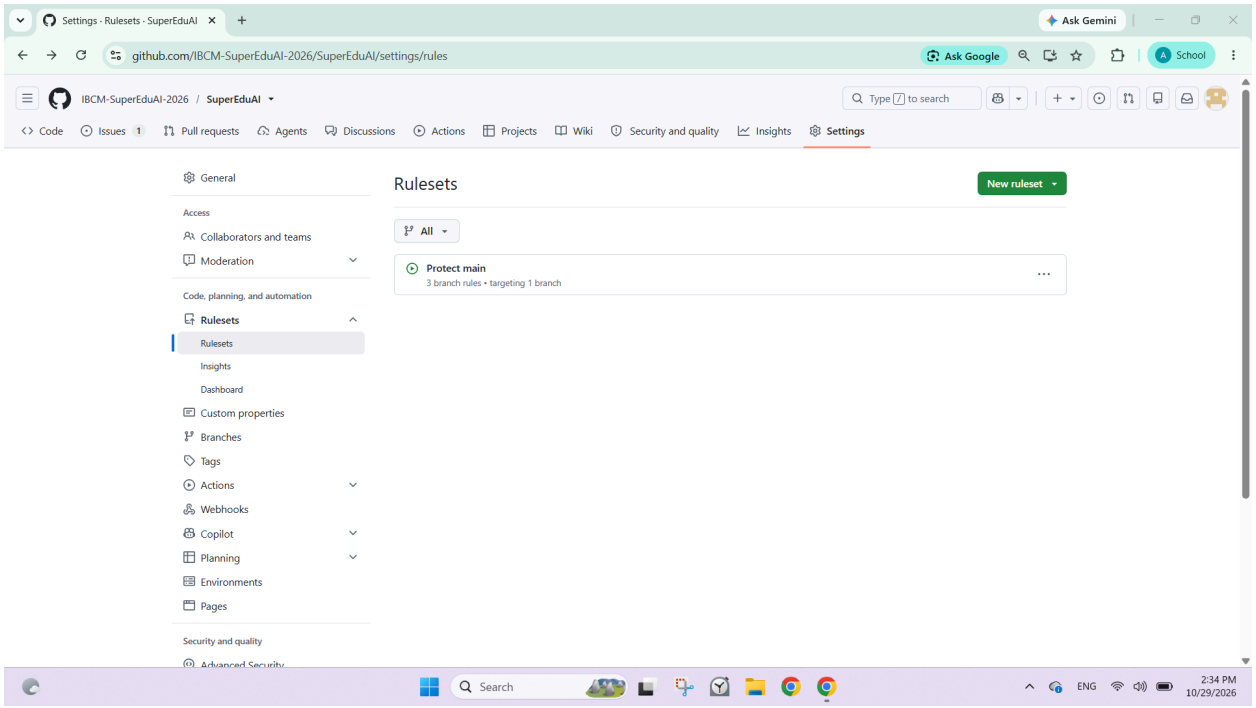


Fig 8: The “Protect main” ruleset, shown as active

This shows a ruleset named **Protect main** with status **Active**, targeting one branch with three rules. A ruleset is an automatic rule that GitHub enforces, so protection doesn’t depend on people remembering to follow it

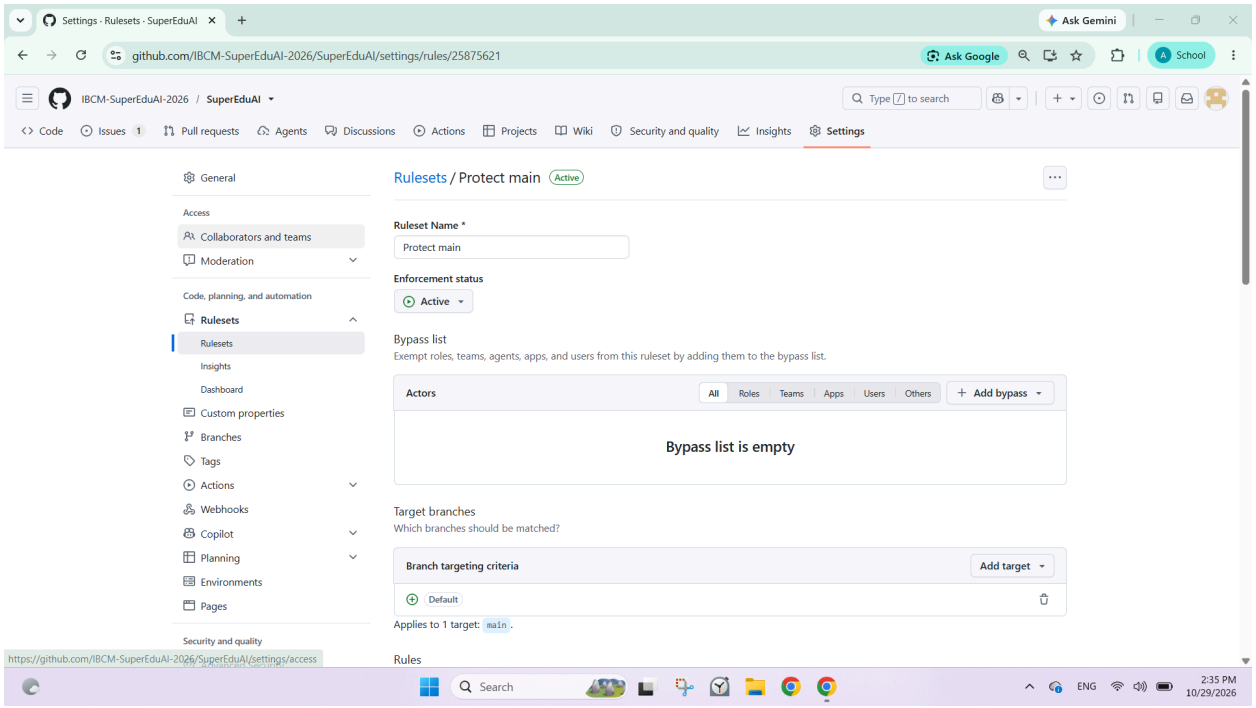


Fig 9a: Ruleset name, enforcement status and bypass list

The enforcement status is Active, so the rules are applied now. The bypass list is empty, which means nobody can skip the rules, including me as the organization owner. I chose this on purpose because a rule that its creator can ignore is weak, and an empty list gives the clearest evidence that everyone follows the same process.

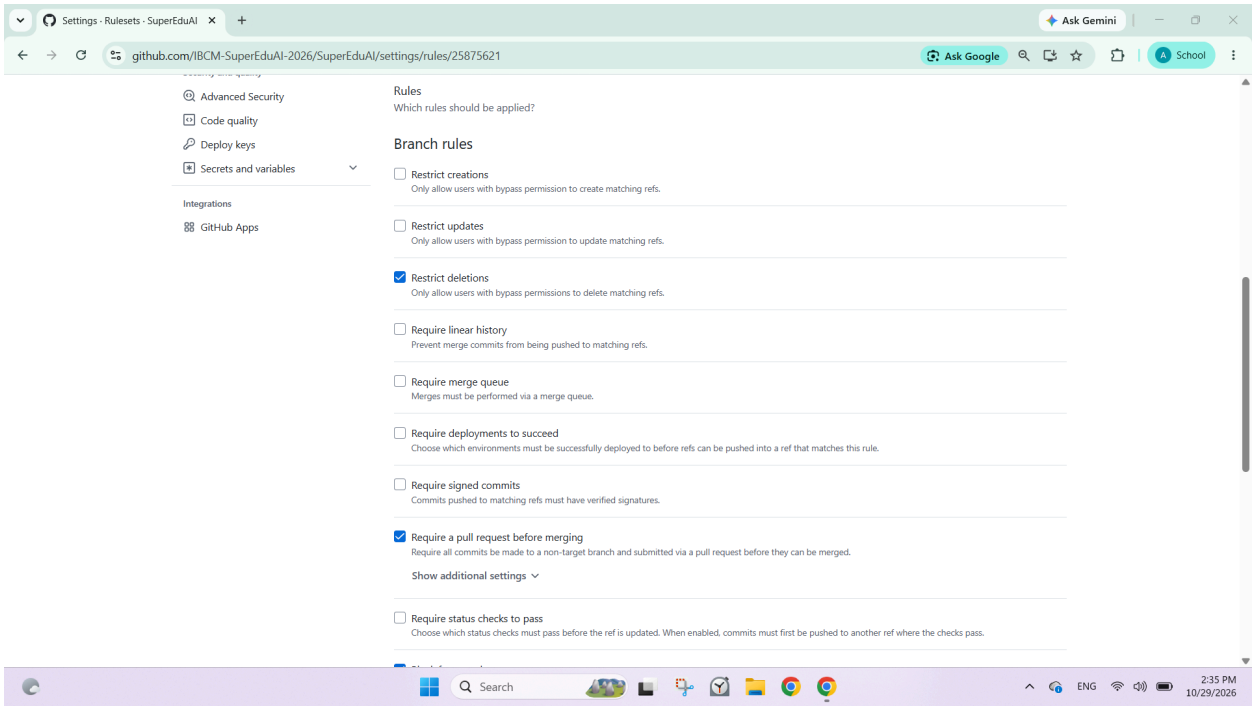


Fig 9b: Target branch of the ruleset

The ruleset targets the **default branch**, which is main. This protects the official version of the project, while working branches stay free for people to use.

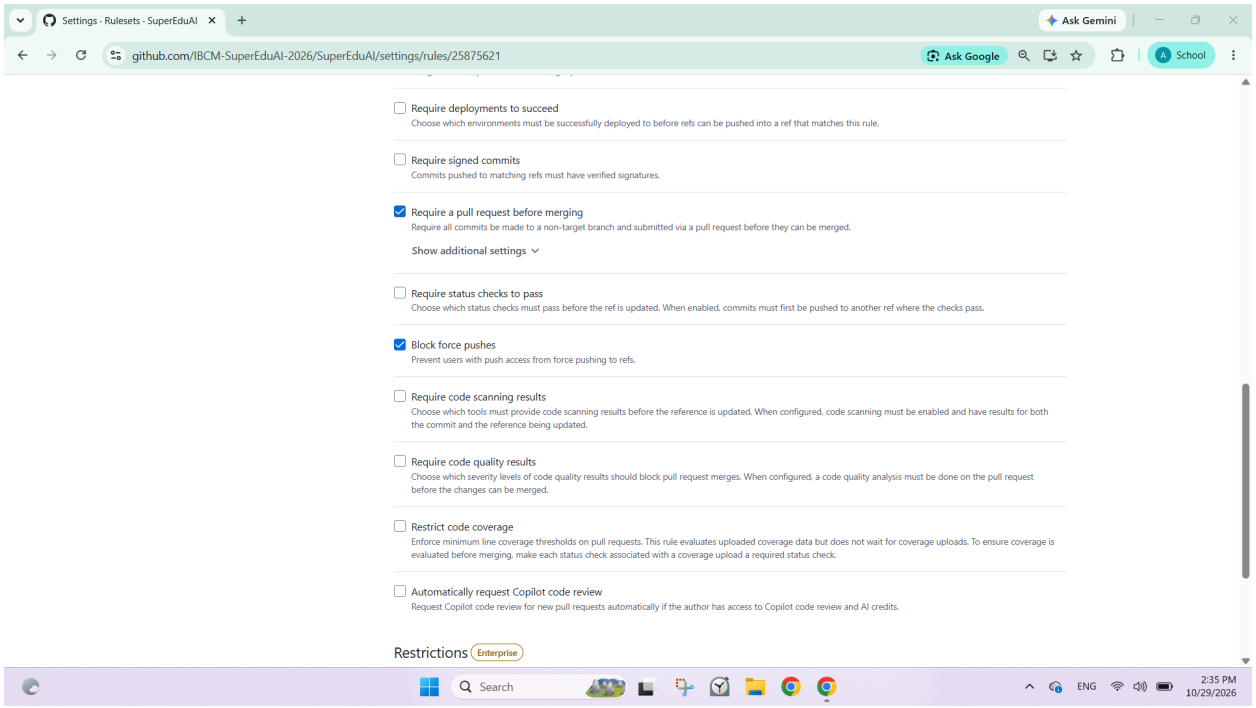


Fig 9c: Rules enforced on `main`, including the pull request requirement

The ruleset requires a **pull request before merging** with **1 approval**. It also blocks force pushes and deletion of the branch. Together these mean nobody can change main directly: every change must be proposed on a branch, reviewed by another person and approved before it is merged.

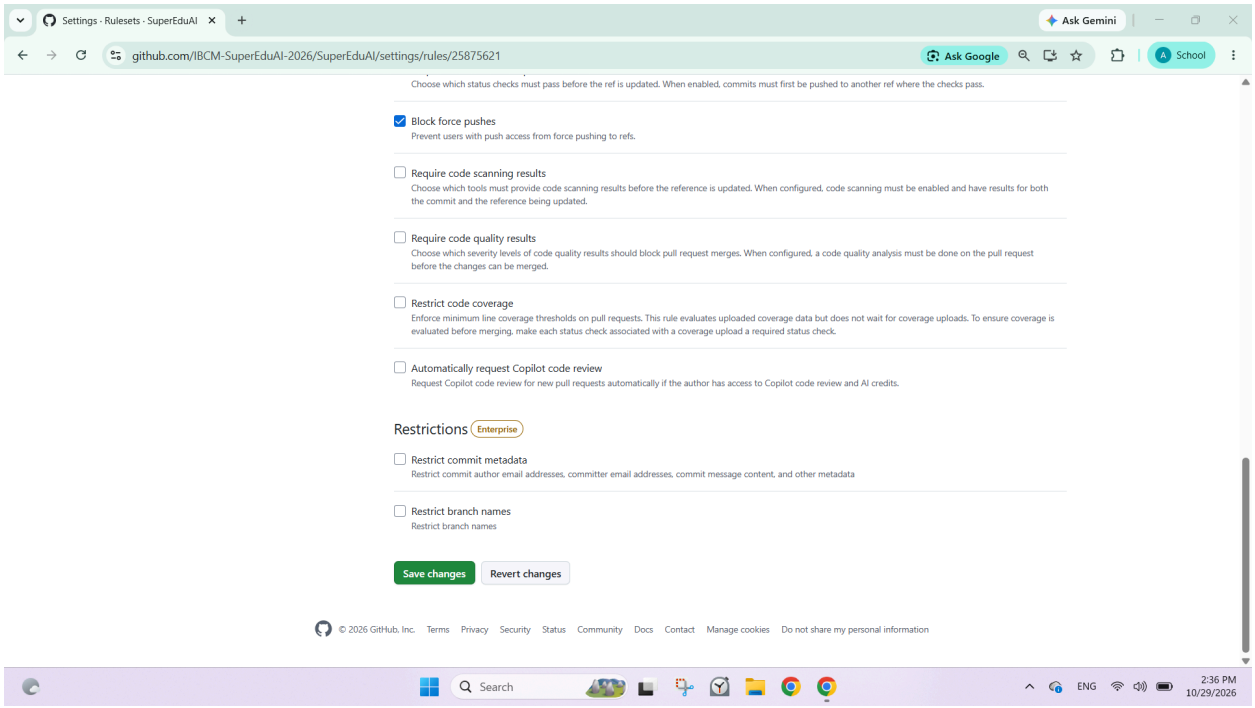


Fig 9d: Rules enforced on `main`, including the pull request requirement

The ruleset requires a **pull request before merging** with **1 approval**. It also blocks force pushes and deletion of the branch. Together these mean nobody can change main directly: every change must be proposed on a branch, reviewed by another person and approved before it is merged.

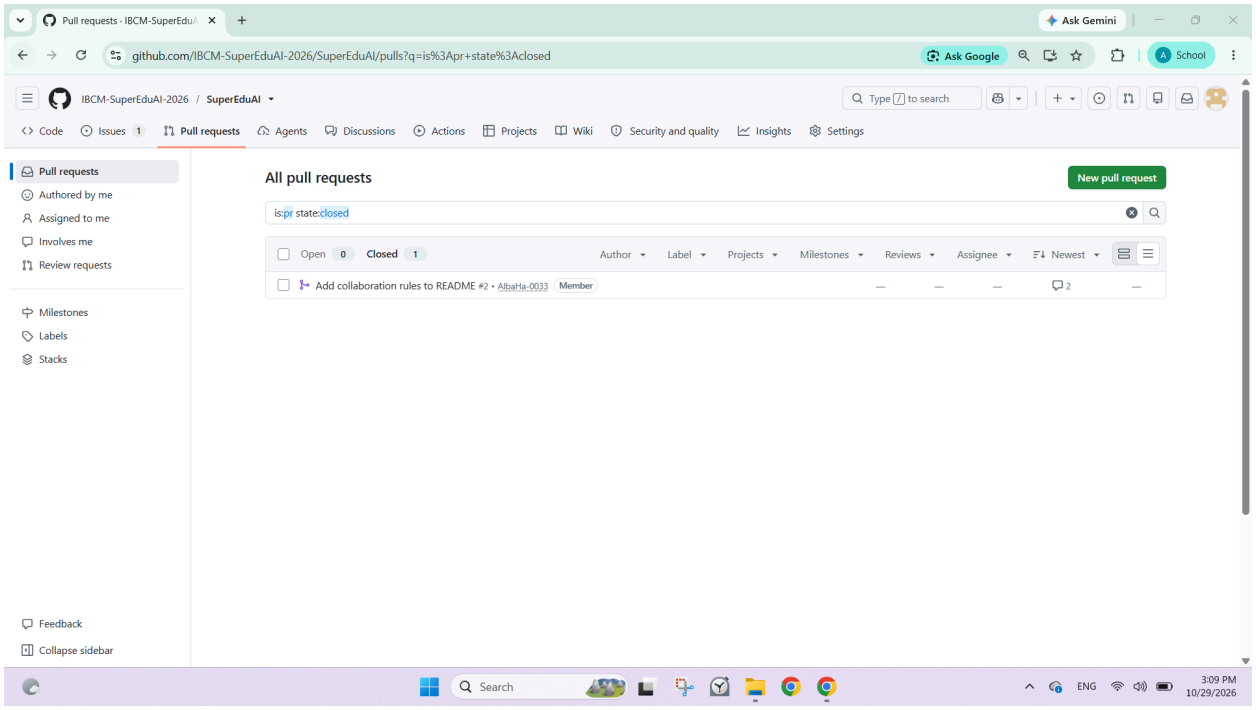


Fig 10a: README pull request with reviewer approval, merged into `main`

To prove the rules work in practice, I changed the README on a separate branch and opened a pull request.The page shows the approval and the “Merged” status, so it is a real example of the review process the project depends on.

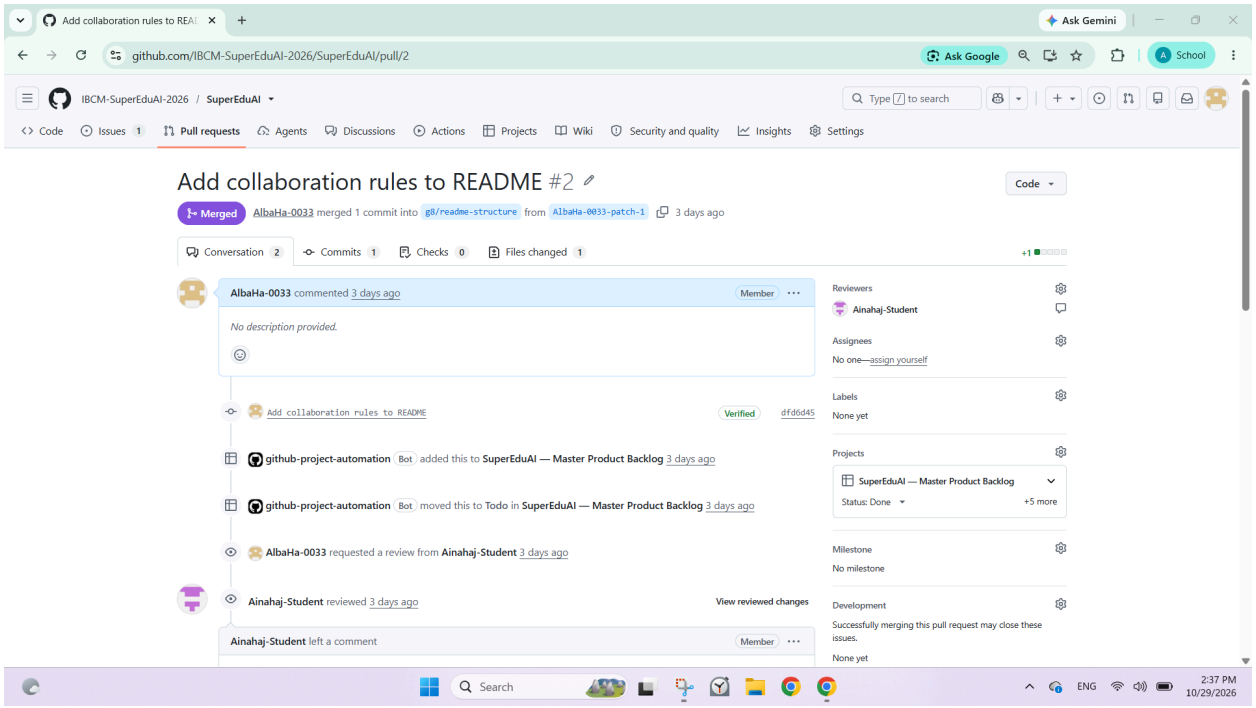


Fig 10b: README pull request with reviewer approval, merged into `main`

To prove the rules work in practice, I changed the README on a separate branch and opened a pull request. AinaHaj reviewed the change and approved it, and then I merged it into `main`. The page shows the approval and the “Merged” status, so it is a real example of the review process the project depends on.

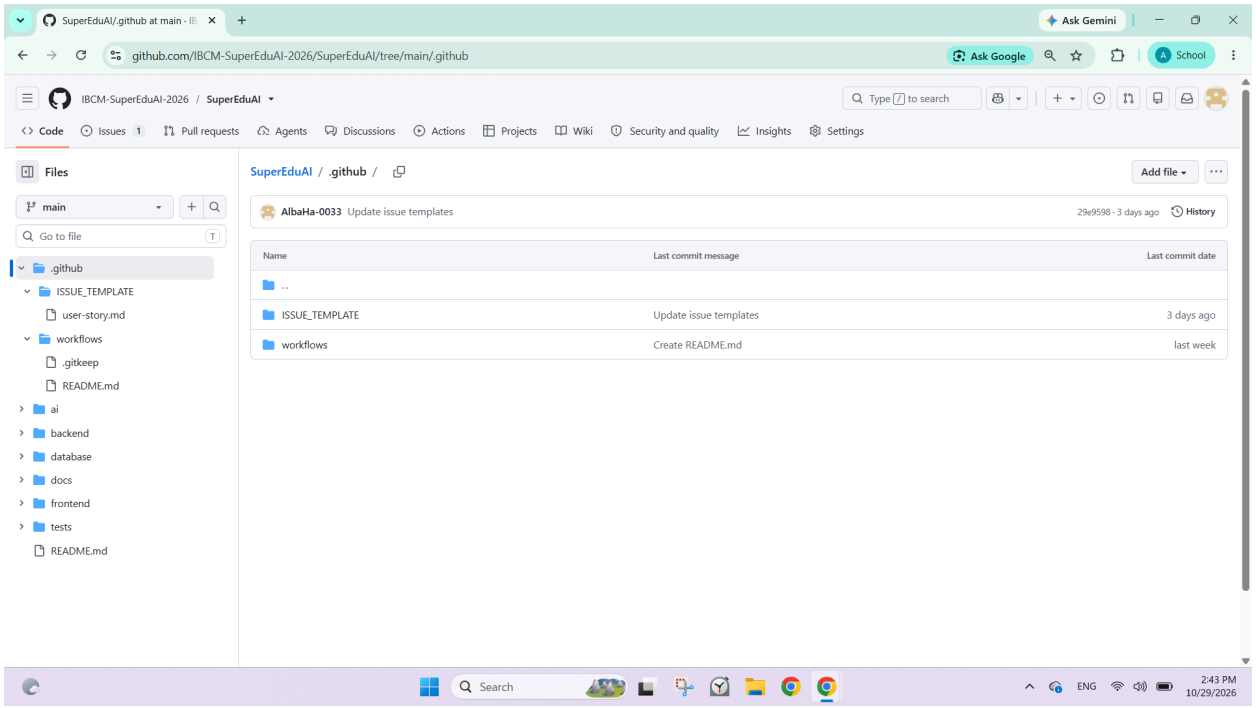


Fig 11: Issue templates in the `.github` folder.

The `.github` folder holds the issue templates for the project. They give every group the same format for reporting work (for example user stories with acceptance criteria), so the nine groups write issues consistently. This folder is also where the automatic build and test workflows of the DevOps group will be added later.

Summary

I created the GitHub organization, the shared repository and the folder structure from the course briefing. I created teams G1 to G9 and invited all members, and I protected the main branch so every change needs a pull request with one approval. This keeps the shared code stable while nine groups work on it.